

Entropy Box: A Knowledge Compiler for Embodied AI

Towards Structured Knowledge Compilation for Robotics Foundation Models

Yuqi Wang

Entropy Box

wangyuqi111and111@163.com • <https://chenli-yy.github.io/entropy-box-public/>

July 30, 2026

Abstract

The bottleneck in embodied AI today is not model capacity — it is knowledge. The engineering knowledge a robot needs already exists: in tens of thousands of papers, GitHub repositories, ROS packages, model hubs, API references, benchmark suites and engineering blogs. But it exists as *prose*, not as structure. A language model can retrieve a passage about inverse kinematics; it cannot tell you, in a form another program can consume, which four distinct engineering paradigms solve the problem, what each one’s inputs, outputs and failure modes are, which open-source assets implement each, and what has to be solved first. Every query re-derives that structure from scratch, and the derivation is discarded the moment the conversation ends.

We argue that robotics needs a **Knowledge Compiler**, not another chatbot. A knowledge compiler ingests unstructured sources once, and emits a persistent, typed, deduplicated, machine-consumable knowledge artifact that improves incrementally and is reused indefinitely. Compile once, execute everywhere.

This paper describes ENTROPY BOX, a working knowledge compiler for robotics and embodied AI. ENTROPY BOX organizes a 2,942-node domain taxonomy (15 top-level domains, 416 sub-domains, 2,511 leaf topics), and compiles each topic into typed *task chains*, *capabilities* and *assets* through a three-phase multi-agent compilation architecture — per-chain closed-loop research and emission, cross-chain merge adjudication, and finalization — whose phases hand off through the file system and share no conversation context. We report the two architectures this replaced and why each failed, and distil the result into a design rule for multi-agent systems: *merge stages when what passes between them is reasoning; split them when what passes between them is an artifact*. Extraction is not trusted: each technical chain is researched by an agent whose output must terminate in an auditable *retrieval ledger* — every query, its hit count, the section it was used in, the candidates that were found and deliberately *excluded*, and script-verified repository metadata — with hard floors on search and fetch volume, below which the document is rejected and re-run. Compilation is ongoing and unattended: at the time of measurement 1,740 of the 2,511 leaf topics had produced a compiled record. The artifact contains 22,873 capabilities and 5,624 assets, linked by 24,529 hierarchy edges, 43,973 ordering edges and 31,803 capability→asset implementation edges, and indexed as 174,189 semantic-view vectors. We define and measure the *amortization factor* — consumptions per compiled unit — as the operative measure of what compilation buys: it currently stands at 1.61, corresponding to 13,936 avoided re-derivations, and, more informatively, it is still rising with coverage rather than saturating. On top of this artifact runs a three-layer hybrid retrieval stack (field-weighted BM25 with 124 robotics synonym groups, Qwen3-Embedding-4B dense retrieval over six per-entity semantic views, and graph expansion) that supports not only search but automatic assembly of executable capability chains with explicit gap annotation.

We describe the architecture, the compilation algorithm, the multi-graph schema and the runtime; walk through a fully-traced compilation of the `a_star_path_planning` topic from raw literature to four distinct executable chains grounded in Nav2, PythonRobotics and four foundational papers; report system statistics; and state plainly what we have not established — notably a head-to-head comparison against query-time baselines, which we argue for but do not claim. We also distinguish our use of “knowledge compilation” from the established tractable-representation sense of Darwiche and Marquis, and ENTROPY BOX from search engines, RAG systems and vector databases. We report two empirical results obtained from the system’s own records: embedding similarity, once it has flagged a candidate duplicate pair, carries too little signal to decide the merge (precision 0.069, AUC 0.629), and no threshold fixes this — which turns a conservative adjudication stage from a design preference into a requirement; and 98.29% of the artifact’s evidence citations resolve to a retrievable document, a metric we argue should be standard for generated knowledge graphs. The artifact is publicly browsable and a read-only retrieval API is open.

Keywords: knowledge compilation, robotics knowledge graph, embodied AI, capability graph, retrieval-augmented generation, provenance, robot foundation models, structured knowledge extraction

1 Introduction

1.1 The knowledge bottleneck

The last three years of embodied AI have been a story of scaling models. Vision-language-action models [4, 10], large cross-embodiment datasets [21], and LLM-based task planners [1] have all improved substantially. Yet the practical experience of building a robot system has barely changed. An engineer asked to make a mobile manipulator pick up a bottle in a cluttered room still spends most of their time on a question that is not about learning at all: *what is the standard decomposition of this problem, what are the established engineering paradigms for each sub-problem, and which existing software actually implements them?*

That knowledge exists. It is in Hart, Nilsson and Raphael’s 1968 paper on A^* [14]; in the Nav2 source tree [19]; in the MoveIt documentation [5]; in a decade of ROS packages [22]; in benchmark leaderboards; in issue threads. What does not exist is a representation of that knowledge that another *program* can consume.

1.2 Why retrieval is not enough

The dominant answer today is retrieval-augmented generation [18, 12]: embed a corpus, retrieve the top- k passages for a query, and let a language model synthesize an answer. RAG is excellent at what it does. But it has three structural properties that make it the wrong primitive for robotics engineering knowledge:

1. **It is amnesic.** Every query re-derives structure from raw text. The expensive act — deciding that “ A^* path planning” actually decomposes into four *non-equivalent* engineering paradigms (grid, hybrid/kinodynamic, jump-point-pruned, any-angle) with different state spaces and different applicability — is performed, used once, and thrown away.
2. **Its output is prose, not structure.** A robot runtime, a planner, or a code generator cannot execute a paragraph. It needs typed entities with declared inputs, outputs, preconditions, constraints and implementations.
3. **It has no global consistency.** The same capability described in twenty documents remains twenty unrelated passages. There is no identity, no deduplication, no hierarchy, no notion that capability X must precede capability Y .

Graph-augmented retrieval [11] improves on the third point by building a graph over a corpus, but the graph is still derived from and bound to one document collection, and is typically rebuilt rather than maintained.

1.3 Knowledge compilation as the missing layer

We propose a different primitive. A **Knowledge Compiler** is a system that takes heterogeneous unstructured sources and produces a *persistent, typed, globally deduplicated, incrementally maintained* knowledge artifact — and does so as a batch, offline, quality-gated process rather than at query time. The analogy to a program compiler is deliberate and load-bearing:

Program compiler	Knowledge compiler
Source code	Papers, repositories, docs, datasets
Lexing / parsing	Evidence retrieval and source parsing
Type checking	Structural validation gate
Symbol table, linking	Global registry with surrogate keys
Optimization passes	Deduplication, relationship mining
Object file / binary	Capability graph + task graph
Incremental rebuild	Content-hash incremental re-embedding
Runtime / loader	Retrieval + agent runtime

The critical property is the same one that makes compilers useful: **the expensive analysis happens once**, its result is a durable artifact, and everything downstream — search, planning, code generation, agent execution — consumes the artifact rather than redoing the analysis.

A note on terminology. “Knowledge compilation” already has an established meaning in AI: the transformation of a propositional theory into a representation (OBDD, DNNF, d-DNNF, ...) in which certain queries become tractable, systematically mapped by Darwiche and Marquis [8]. We are using the term in a different, engineering-systems sense: the compilation of *unstructured natural-language and code sources* into a typed, queryable knowledge artifact. The two senses share the essential intuition — pay an offline cost once to make online queries cheap and well-defined — but they are not the same construction and we do not claim any of the tractability results of the classical line of work. Where ambiguity matters we write *source-level knowledge compilation*.

1.4 Contributions

We state our contributions as named mechanisms rather than as a system description, and we separate what we *propose* from what we *demonstrate*.

1. **Source-level knowledge compilation, as a paradigm.** We propose moving the structural analysis of engineering knowledge out of the request path and into an offline, gated, incremental compilation stage whose output is a persistent typed artifact (§2). We position this against query-time derivation (RAG, GraphRAG, agentic research pipelines) and distinguish it explicitly from classical knowledge compilation in the OBDD/d-DNNF sense [8]. *This is a position, argued and partially quantified, not an empirically settled result* — see the scoping statement in §2 and the limitations in §11.
2. **Amortization as the operative measure of compilation value.** We define the amortization factor $\alpha = |U|/|C|$ — consumptions per compiled unit — and argue it is the quantity the “compile once” claim is actually about. We measure it on a live artifact ($\alpha = 1.61$; 13,936 re-derivations avoided) and, more importantly, show that it is a *function of coverage* that has not saturated: compilation becomes a better deal as the domain fills in (§2.3).
3. **The Retrieval Ledger: an auditable evidence regime for LLM-generated knowledge.** We define a provenance discipline in which every generated unit of knowledge must be accompanied by a machine-checkable record of the retrieval that grounded it: enforced search and fetch floors, per-query accounting of where each result was used, script-verified repository and paper metadata, and — the load-bearing element — *mandatory reporting of candidates found and deliberately excluded, with reasons* (§5.11). We argue that admission control and provenance, not prompt engineering, are the substantive problems in LLM-based knowledge graph construction, and we borrow the discipline from systematic-review methodology rather than inventing it.
4. **A multi-agent compilation architecture, and the design rule behind it.** We present a three-phase architecture in which each technical chain is compiled by a single closed-loop agent while orchestration is split across separate agent sessions handing off through the file system (§5). We report the two architectures this replaced and the specific reason each failed, and generalize the outcome into a rule we believe applies beyond this system: *merge two stages into one context when what passes between them is reasoning; split them when what passes between them is an artifact* (§5.2). We also identify two sequencing constraints — normalize entity identifiers before measuring chain overlap, and rekey to surrogate identifiers only after all semantic adjudication is complete — in which a locally reasonable order silently corrupts a later decision (§5.6).
5. **A multi-graph intermediate representation with semantic views.** We present a representation in which capability specialization, capability ordering, capability implementation, provenance and literature grounding are *separate typed edge families* over a shared entity space, and in which each entity is embedded as several independent facets rather than one blended vector — three of which are synthesized from graph structure at zero generation cost (§6). We show this representation is what makes intent-conditioned retrieval, typed graph expansion and gap-annotated chain assembly expressible at all (§7).
6. **Two empirical results about LLM-built knowledge graphs.** Using the system’s own adjudication records as labelled data, we show that embedding similarity, *conditional on having flagged a pair*, is far too weak to decide duplication (precision 0.069, AUC 0.629 with CI [0.531, 0.722]), and that raising the threshold cannot fix it — so a conservative adjudication stage is a requirement rather than a safety margin (§9.1). We also propose *provenance resolvability* — the fraction of an artifact’s own evidence citations that resolve to a retrievable document — as a cheap, hard-to-game health metric for generated knowledge graphs, and report 98.29% on ours (§9.2).

7. **An open, traced instantiation.** ENTROPY BOX is the first implementation of the paradigm, not the paradigm itself. We report a fully traced compilation of a real topic from literature to four non-equivalent executable chains (§8), system-wide measurements on a live artifact of 22,873 capabilities and 5,624 assets (§9), a public browsable artifact and open retrieval API, and a candid account of what does not yet work (§11).

2 Why Compile? The Amortization Argument

Before describing the system we should answer the question a reader is entitled to ask first: *what does compilation buy that query-time derivation does not?* This section makes the argument, states its scope, and quantifies it on the live artifact.

2.1 Where the work happens

Figure 1 contrasts the two arrangements. In query-time systems — naive RAG, GraphRAG, agentic research pipelines, tool-calling stacks — the structural analysis sits *inside* the request path. The retrieved passages are raw; deciding that A^* decomposes into four non-equivalent paradigms with different state spaces is work the model performs during the request, and that work is discarded when the response is returned. In a compiler, the analysis sits *before* the request path and its result is a durable object.

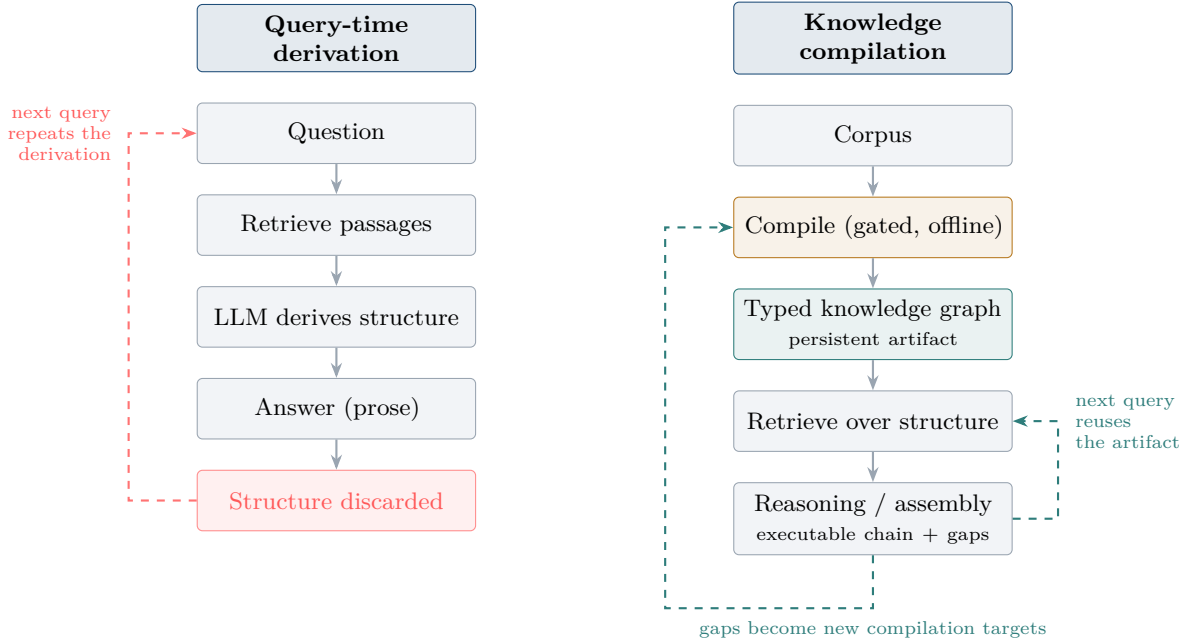


Figure 1: The structural difference. A query-time system performs the expensive analysis inside the request and discards it; a compiler performs it once and serves the result. Slogan form: *traditional retrieval compiles on every query; a knowledge compiler compiles once.*

2.2 What follows from this, and what does not

Three consequences follow directly, and we state them as claims we can defend:

1. **Redundant derivation is eliminated, and the saving grows with corpus size.** If n distinct queries or topics all depend on the same underlying analysis, a query-time system performs it n times; a compiler performs it once. §2.3 measures n on the live artifact.
2. **The output is consumable by programs, not only by readers.** A passage cannot be type-checked, composed or executed. A typed capability with declared inputs, outputs and implementations can be. This is what makes chain assembly with explicit gap annotation possible at all (§7); no amount of retrieval quality produces it.

3. **Consistency becomes enforceable.** Query-time derivation has no global identity: the same capability can be described differently on two consecutive queries, and nothing detects it. A compiler has a registry, so identity, deduplication and cross-topic linkage are decidable properties rather than emergent ones.

We are equally explicit about what does *not* follow. We do **not** claim that compilation yields higher answer accuracy than a strong RAG or GraphRAG baseline, or lower hallucination rates, or better latency. Those are empirical questions, they require a benchmark that does not currently exist for robotics engineering knowledge, and we have not run them (§11). Nor is compilation free: it moves cost from query time to build time, and that cost is only worth paying when the same knowledge is consulted repeatedly — which is precisely the condition §2.3 tests.

Relation to adjacent systems. The distinction is about *where the structure lives*, not about which components are used. GraphRAG builds a graph, but derives it from and for a specific corpus, typically rebuilding rather than maintaining it, and its nodes are text-derived entities rather than typed engineering objects with declared interfaces. Agent frameworks and tool protocols such as MCP standardize *access* to knowledge; they say nothing about whether the knowledge behind the interface is structured or persistent — in fact a compiler is a natural thing to put behind such an interface, and we do (§7). Deep-research products perform excellent query-time synthesis and discard it. None of these are competitors to compilation; they are consumers of, or complements to, the artifact it produces.

2.3 Measuring amortization

The claim “compile once” is only meaningful if knowledge is in fact reused. We measure this directly. Let C be the set of distinct compiled capabilities and U the set of (capability, topic) usages. A query-time system that derived structure per topic would perform $|U|$ derivations; the compiler performs $|C|$. We call

$$\alpha = \frac{|U|}{|C|}$$

the **amortization factor**: the average number of times each compiled unit is consumed.

On the live artifact, $|C| = 22,777$ and $|U| = 36,713$, giving $\alpha = 1.61$. In absolute terms, **13,936 re-derivations are avoided**. The distribution is heavy-tailed: 6,533 capabilities (28.6%) are used by more than one topic, and the most reused unit — *PDDL Problem Instance Generation* — is consumed by 32 distinct topics, meaning a query-time system would re-derive it 32 times.

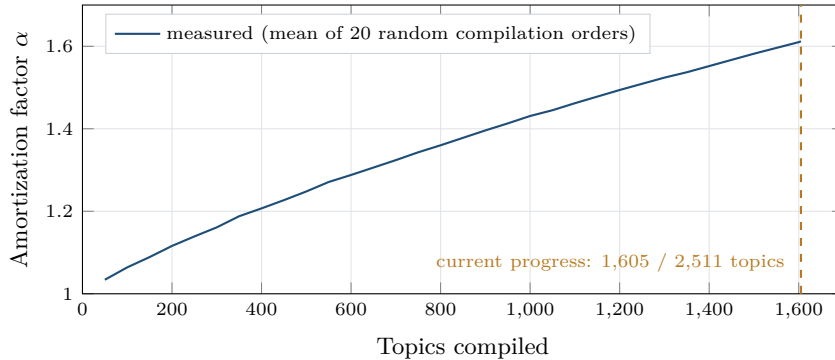


Figure 2: Amortization factor as a function of how much of the corpus has been compiled, averaged over 20 random compilation orders (variance across orders is negligible at this scale). The curve is still rising at +0.029 per 100 topics in its final quarter and shows no sign of saturation: the marginal value of compiling one more topic is currently *increasing*, because each new topic is more likely to reuse existing capabilities than to mint fresh ones. Roughly 36% of the corpus remains uncompiled.

Figure 2 shows the more informative result. Amortization is not a constant of the system — it is a function of coverage. At 50 compiled topics $\alpha \approx 1.03$ (almost nothing is shared yet); at 1,605 topics $\alpha = 1.61$, and the curve has not flattened. This is the property that makes compilation a sensible investment rather than a one-off cost: the artifact becomes a better deal the more of the domain it covers, because a capability minted for one topic is available to every topic compiled afterwards.

Scope of this measurement. We are measuring *structural* reuse within our own compilation process — how often a compiled unit is referenced by more than one topic. This is a lower bound on, and not the same as, the derivation savings a deployed query-time system would incur, which depend on query distribution. It also does not speak to answer quality. We report it because it is the quantity the “compile once” claim is actually about, and because it is measurable without a benchmark; we do not stretch it further than that.

3 Related Work

Robot knowledge bases. The idea that robots should share structured knowledge has a long lineage. KnowRob [26] pioneered a description-logic knowledge processing system tying symbolic action descriptions to perception and control, and CRAM [3] showed how such knowledge drives executable plans. RoboEarth [28] proposed a shared web database of action recipes, object models and maps. RoboBrain [25] aggregated multimodal robotics knowledge into a large graph. IEEE 1872-2015 [15] standardized a core ontology for robotics and automation. ENTROPY BOX differs from all of these in *what it compiles from* and *what it compiles to*: prior systems largely encode hand-authored ontologies or execution traces, whereas ENTROPY BOX compiles the open engineering literature and open-source ecosystem into an engineering-decision graph (which paradigms exist, what each needs, what implements it) rather than an execution-semantics ontology. The two are complementary: an engineering-decision graph tells you *which* grasp planner to use; an execution ontology tells the robot *how* to run it.

Scientific and open knowledge graphs. Wikidata [27] and DBpedia [2] demonstrated large-scale collaborative and extracted knowledge graphs; the Open Research Knowledge Graph [16] argued for structured, machine- actionable representations of scholarly contributions. ENTROPY BOX shares ORKG’s premise that papers should become structured records, but targets a single vertical (robotics/embodyed AI) at much finer granularity: not “this paper reports result *R*” but “this paradigm requires these inputs, produces these outputs, fails under these constraints, and is implemented by these repositories.”

Retrieval. Our runtime builds on standard IR components: BM25 [24], dense passage retrieval [17], cross-encoder reranking [20] and reciprocal rank fusion [6], with Qwen3-Embedding [23] as the encoder. Our contribution here is not a new retrieval algorithm but the observation that retrieval over a *compiled, typed* corpus admits structure-aware refinements — per-facet semantic views with intent-conditioned weighting, and graph expansion along typed edges — that are unavailable when retrieving over raw text.

Multi-agent orchestration and context engineering. Decomposing a task across multiple LLM agents is now routine, and the usual justification is capability specialization — give each agent the role it is best at. Our experience (§5.1) points at a different and, we think, more binding constraint: the decisive question is not what each agent is good at but *what can survive the handoff between them*. Our second architecture failed precisely because it decomposed along role boundaries, and roles turned out to be the wrong seam — the seam that mattered was whether the inter-stage payload was an artifact or a line of reasoning. We are not aware of this being stated as a design rule, and we offer it as one (§5.2). Relatedly, we treat agent context as a budget to be allocated by the orchestrator rather than as an accumulating log, and agent liveness as a systems concern with heartbeats and enforced timeouts (§5.9) rather than as an exception path.

LLM-based knowledge extraction. Using LLMs to construct knowledge graphs is now common practice. The recurring failure mode is quality: fluent output that is structurally invalid or globally inconsistent. Our position is that the interesting engineering is not the extraction prompt but the *admission control and provenance discipline* around it — which is why §5 devotes most of its length to gates and evidence ledgers rather than to generation. The closest analogue we are aware of is systematic-review methodology in evidence-based medicine, where recording the search strategy, the databases queried and the studies excluded with reasons is what separates a review from an opinion. We adopt the same discipline for machine-generated knowledge, for the same reason.

4 System Architecture

ENTROPY BOX is organized as a strict left-to-right pipeline from raw sources to robot, with a persistent compiled artifact in the middle (Figure 3). Everything to the left of the artifact runs offline, in batch, and

is allowed to be slow and expensive. Everything to the right runs online and consumes the artifact without re-deriving it.

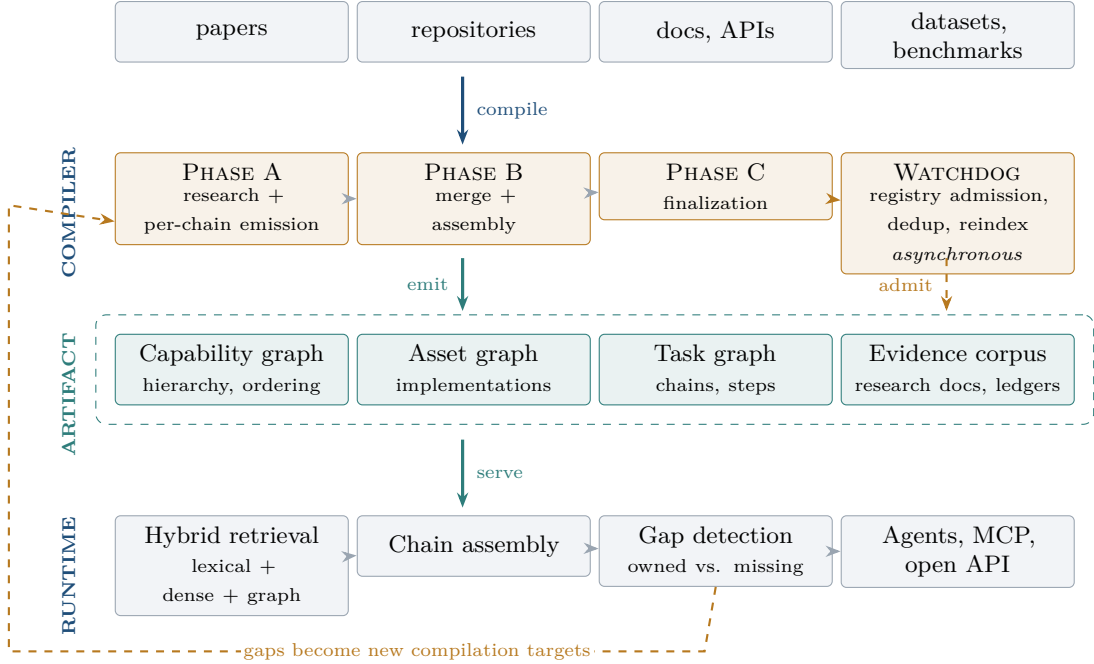


Figure 3: ENTROPY BOX end to end. The compiled artifact (dashed box) is the durable product: everything above it is an offline batch compiler, everything below it is a consumer. Registry admission and index maintenance run asynchronously in a separate watchdog, outside the compilation transaction (§5.8). Gaps found at runtime return as new compilation targets.

4.1 The taxonomy backbone

Compilation is not undirected crawling. It is driven by a manually curated three-level taxonomy that defines *what should exist*, so that coverage is measurable and boundaries between topics are explicit. The current tree has 2,942 nodes:

Level	Meaning	Count
L0	Top-level domain	15
L1	Sub-domain	416
L2	Compilation unit (topic)	2,511
Total		2,942

The 15 L0 domains are: Foundation Models, Human-Robot Interaction, Learning, Localization, Manipulation, Mapping, Motion & Control, Multi-Robot Systems, Navigation, Perception, Planning, Reasoning & Agent, Safety/Trust & Security, Simulation & Digital Twin, and System Infrastructure.

The L2 node is the **unit of compilation**: one L2 topic (for example `a_star_path_planning` or `finger_gait_planning`) is compiled in one transaction, producing one topic record containing several task chains and the capabilities and assets they reference. Sibling L2 nodes are passed to the extractor as explicit *boundary constraints*, which is how the system prevents adjacent topics from absorbing each other’s content — the compiler’s equivalent of scoping rules.

5 Knowledge Compilation

This section describes the compiler. We lead with its architectural history, because the current design is the third attempt and the two failures are more instructive than the survivor.

5.1 Two failed architectures

v1 — one agent, one topic, one context. The original compiler gave a single agent the whole topic: research everything, then write the whole record. It failed in three ways that turned out to be the same failure. Research depth was squeezed by context length — as the conversation filled with retrieved material, later chains got shallower treatment than earlier ones. Information from different technical paradigms contaminated each other, so chains converged toward a common vocabulary that belonged to none of them. And by the time the agent wrote the record, a large fraction of fields were *drafted from memory* rather than grounded in anything retrieved; the research documents degenerated into conclusions with no recoverable evidence behind them.

v2 — one agent per chain, split into a relay. The obvious fix was to give each technical chain its own context. We did that, and further split each chain across three specialist agents in sequence — research, then asset verification, then JSON emission — with all chains completing research before any chain entered generation. This version failed for a reason we did not anticipate and now consider the most important lesson in the project:

Design intent evaporates at the handoff. Why this chain has *this* step sequence; why step 4 must follow step 3; why two capabilities were not split apart — these are judgments formed *during* research. Only their conclusions can be written into a document. The reasoning that produced them cannot.

The downstream agent therefore had no choice but to transcribe the document literally. Two consequences followed. First, when the document did not cover a field, the JSON agent had no recourse: it was forbidden from searching (without research context, a searching agent fabricates from memory), so a gap that one query would have closed had to wait for human intervention or an entire additional session. Second, the asset-verification agent could not feed back: on discovering that a capability had no real implementation anywhere, it had neither the authority nor the context to revise the capability list the research agent had designed. It could only execute.

5.2 The principle: split on artifacts, merge on reasoning

The resolution generalizes beyond this system, and it is the design rule we would most want a reader to take away:

**Merge two stages into one context when what passes between them is reasoning.
Split them into separate contexts when what passes between them is an artifact.**

Reasoning does not survive serialization; artifacts do. A handoff that requires the receiver to reconstruct *why* is a handoff that should not exist. A handoff that requires the receiver only to consume *what* is a handoff that should be made explicit, durable and resumable.

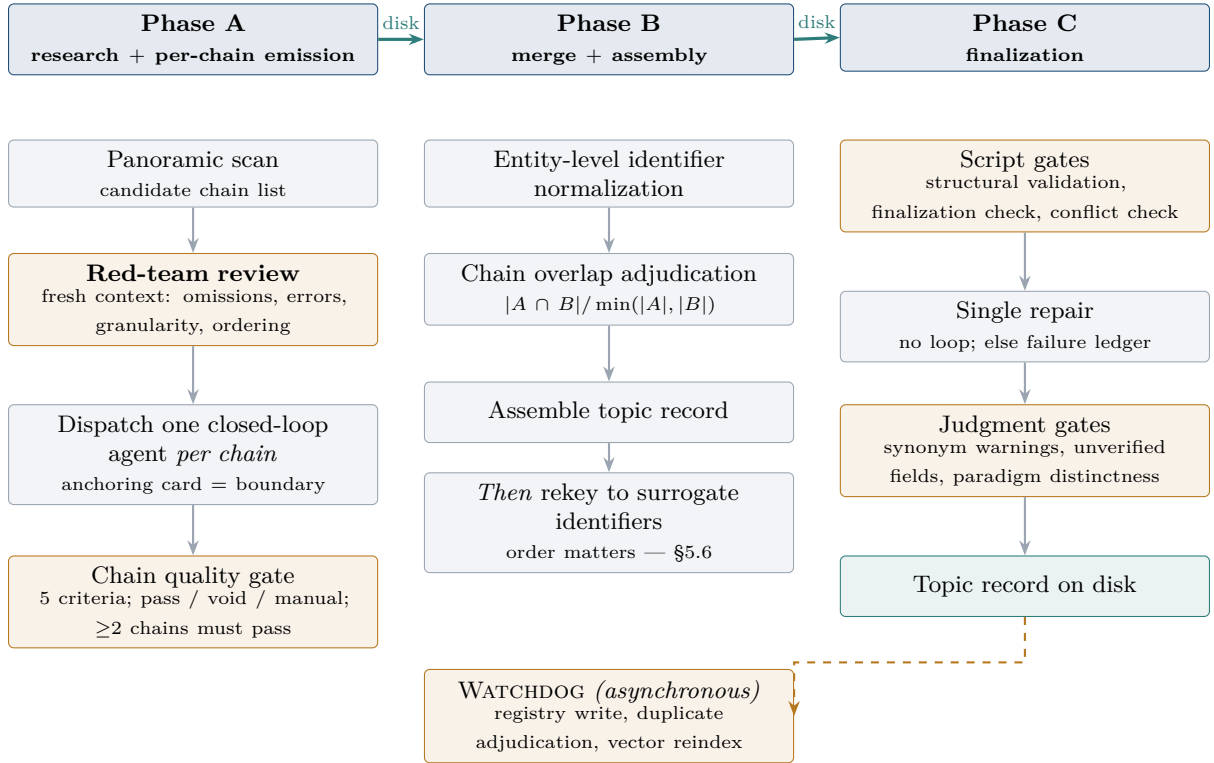
Applied to this compiler, the rule cuts in two directions at once, which is why the resulting architecture looks inconsistent until the rule is stated:

- **Within a chain, merge.** Research, asset verification and record emission happen in *one* agent session, because what passes between them is design intent.
- **Across the orchestration, split.** Panoramic scan, cross-chain merge, and final validation happen in *three separate* agent sessions handing off through the file system, because what passes between them is finished artifacts — documents, fragments, a record.

5.3 v3: chain-level closed loop under a three-phase orchestrator

Figure 4 shows the resulting architecture.

The chain-level closed loop. One agent owns one chain end to end: segment A researches the chain’s structure, segment B verifies its assets, segment C emits the JSON fragment — all in one session, then the session exits and its context is released. Because the agent that writes the record is the agent that did the research, gaps discovered during emission can be closed by a few additional queries on the spot, and segment B can revise decisions made in segment A. The agent reads one specification file per segment



No agent in phases A–C may write the registry or the index.

Figure 4: The three-phase compilation architecture. Phases hand off through the file system and share no conversation context; within phase A, each chain is compiled by a single closed-loop agent. Registry writes and index maintenance are performed asynchronously by a separate watchdog and are explicitly outside the compilation transaction.

rather than loading all three up front, since pre-loading would spend the context the closed loop was meant to save.

The cost of this design is that chains run in parallel and mint identifiers independently, so the same real capability reliably acquires two identifiers under two names. That cost is paid, deliberately and in one place, by Phase B. The trade is favourable because *merging is mechanically decidable* — it has criteria, thresholds and a written adjudication standard — whereas design intent, once lost, cannot be reconstructed at any price.

A secondary benefit is scheduling: agents per topic fall from $3N$ to N . A five-chain topic went from fifteen dispatch/recover cycles to five, reducing both orchestration overhead and the number of opportunities for a node to hang.

Why the orchestrator still splits into three. The closed loop solves context pressure for a chain but not for the controller, which would otherwise accumulate breakpoint inventory, service checks, the panoramic scan, N rounds of dispatch and recovery, per-chain gating, cross-chain merge adjudication, assembly, rekeying and final validation in one context. Crowding would be worst exactly at merge and final validation — the two stages that most need cold judgment. Splitting is possible only because resumability already required every intermediate product to be identifiable and reusable on disk; the phase boundary is an existing internal breakpoint promoted to an inter-agent handoff.

5.4 Phase A: research and per-chain emission

Boundary contract. Every dispatched agent receives, verbatim and inside its fence, the topic’s *anchoring card*: the scope statement, the sibling-topic list and the explicit exclusions. Paraphrasing it or referring to it by path is forbidden. This is the mechanism that stops adjacent topics from absorbing each other’s content; a chain agent that finds material belonging to a sibling records a cross-reference rather than expanding into it.

Red-team review. After the panoramic scan produces a candidate chain list, that list is submitted to a *fresh conversation* for adversarial review against four questions: what important technical routes are missing; which chains have mis-assigned paradigms, wrong representative systems or wrong citations; whether any chain’s granularity is too coarse or too fine; and whether the ordering is defensible. The conclusions are written into the research document and the candidate list is revised before any chain is dispatched.

Two properties make this worth its cost. It runs in a clean context, so it is not anchored by the reasoning that produced the list. And it runs at the cheapest possible moment: merging two near-duplicate candidate chains here costs one edit, whereas discovering the duplication after both have run full closed-loop compilation costs two wasted agent sessions and a merge adjudication.

Evidence floors and the retrieval ledger. Within a chain agent, segment A must perform at least 10 web searches and 8 page fetches, and segment B at least 6 and 8 respectively, plus registry lookups and repository verification calls. Every one is logged. §5.11 describes the ledger format, including its mandatory negative-results table.

Context-budget discipline. When a chain agent returns, the controller records *one line* — chain number, verdict, entity counts, whether gaps were flagged — and does not read the returned artifacts into its own context. This is explicit in the specification: the controller’s context is reserved for dispatch and gating decisions. Treating context as a budget to be allocated, rather than as a place where information accumulates, is what makes a 30-topic batch tractable.

Recovery gates and non-binary verdicts. Each returned chain is validated twice by script — once for the research document, once for the fragment — and both must report zero errors. The controller then applies a five-criterion quality rubric (Table 1). Verdicts are deliberately *not* binary: a chain may pass, be voided, or be marked *manual intervention* after two failed reruns. Voided chains keep their fragments on disk for the record but are marked “not to be assembled”. A chain that cannot be settled does not block the batch.

If fewer than two chains pass, the controller returns to the panoramic scan rather than lowering the bar — a topic with one paradigm is a signal that the scan was incomplete, not that the topic is simple.

Table 1: Phase-A single-chain quality rubric. Note that “negative evidence” — routes and assets that were investigated and rejected — is a graded criterion, not an optional courtesy.

Criterion	Pass condition	On failure
Real-system support	chain maps to an identifiable engineering system or paper	void the chain
Step sufficiency	≥ 4 sub-steps (phase nodes do not count)	redispatch or void
Dataflow coherence	each step’s inputs connect to prior outputs; no magic steps	redispatch
Derivation completeness	the five design questions are all answered and coherent	redispatch
Negative evidence	investigated-but-rejected routes and assets are recorded	warning only

Findings, not decisions. Phase A closes with a cross-chain registry spot-check: does the same real project resolve to the same registry identifier in every chain that used it? Discrepancies are recorded as *leads for Phase B* and explicitly not acted upon, because a controller that starts merging here would collide with the phase that owns merging.

5.5 Phase B: merge and assembly

Phase B is the only point in the entire flow with simultaneous visibility of every finished chain, and it does its own work rather than delegating — the judgment required is exactly what would be lost by dispatching it. It is correspondingly the phase we allocate the strongest available model to.

Entity-level normalization. Every cross-chain pair of similar entities is adjudicated same/different under a written standard shared with the global deduplication process. Assets carry a hard rule: two assets whose sources are different repositories are different, however similar their descriptions, and independently installable sub-packages of a monorepo are distinct assets. When merging, the surviving identifier is the one already present in the global registry if either is; the discarded entity’s unique applicability and constraint text is folded into the survivor before the merge, not after. Uncertain pairs are left unmerged, on the standing argument that a false merge destroys a distinction irreversibly while a false retention is corrected later at no loss.

Chain overlap adjudication. With identifiers normalized, pairwise chain overlap is computed over capability sets only:

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}.$$

Assets are excluded deliberately — two chains both using PyTorch are not the same chain. Overlap ≥ 0.70 merges the pair; $[0.50, 0.70)$ is a grey band requiring the adjudicator to read both chains’ research documents and decide on the basis of whether they target the same real system or family; below 0.50 both are kept. Merged chains inherit a provenance sentence recording which chains were combined, at what overlap, and under which criterion — and, importantly, *both* original research documents remain referenced by the assembled record, because dropping one would delete that paradigm’s entire research corpus from the evidence index.

5.6 Two ordering constraints

Two sequencing rules in Phase B are, in our view, the most transferable technical findings in the system. Both are cases where a locally reasonable order silently degrades a decision made later.

Normalize entities before measuring chain overlap. If the same capability still carries two identifiers in two chains, it counts as non-overlapping, and the overlap ratio is systematically *under*-estimated — precisely for the chain pairs that are most redundant, because those are the pairs with the most shared capabilities and therefore the most duplicate identifiers. Highly overlapping chains would then never be merged. The bias is not random noise; it is anticorrelated with the thing being measured.

Rekey identifiers after merging, never before. Capability identifiers are opaque surrogate keys (§5.13), which is the right representation for storage. But the merge decision is a semantic judgment made by reading names and descriptions. Rekeying first would present the adjudicator with two hexadecimal strings and no way to see that they denote the same thing; the specification’s own phrasing is that adjudication quality falls off a cliff.

The general lesson is that surrogate keys are not uniformly good. They are correct for reference and storage and actively harmful in any stage whose input is human-or-model semantic judgment. A system that uses them must therefore be explicit about *when* the transition from names to keys occurs — and it should occur as late as possible, after every stage that needs to read.

5.7 Phase C: finalization

Phase C validates and does not generate. Script gates run first: structural validation, a finalization checker, and a registry conflict check in `--check` mode. The finalization checker deliberately partitions its own output into *blocking* findings, which it asserts have no false positives and must be fixed, and *informational* findings, which it acknowledges do have false positives and are passed to judgment rather than enforced. We regard this self-classification as good practice: a checker that does not distinguish what it is certain about from what it is guessing at will eventually be ignored wholesale.

Repair is dispatched *once*. If the gates still fail, the topic goes to a failure ledger for human attention rather than into a repair loop.

Phase C also carries an explicit *time-discipline* instruction that we think is underappreciated in agent system design: it is forbidden to re-read the chain research documents in full, to re-verify asset URLs, or to redo the adversarial reading already performed upstream. The stated reason is that the acceptance records of those earlier steps *are* the evidence, and repeating them consumes budget without improving quality. Without such an instruction, a capable agent given a validation role will re-derive everything it can, and a pipeline that re-validates at every stage does not converge.

One judgment rule from this phase deserves quoting on its own, because it inverts the usual optimization pressure: where a field could not be verified, the agent must leave it marked unverified, and *fabricating a citation to make the unverified-field count go down is treated as a far more serious failure than leaving the field unverified*.

5.8 Registry writes are outside the transaction

No agent in Phases A–C may write the global registry or the vector index. The orchestrator and all task nodes are prohibited from invoking registry synchronization other than in check mode, and from any index write. Real registry admission, duplicate adjudication and index maintenance are performed asynchronously by a separate watchdog process; the orchestrator’s only post-processing step is a version-control commit.

This is a meaningful correction to the naive picture of a compiler in which linking is the last stage of each translation unit. Here, per-topic compilation and global registry admission are decoupled: 30 topics compile concurrently without contending for the registry, and a registry problem cannot stall compilation. The cost is that the invariant “everything in the registry is structurally valid” is enforced at admission by the watchdog rather than by the compiling agent, and that the registry lags the topic records by the watchdog’s period.

5.9 Liveness, idempotence and failure

Compilation runs unattended and continuously, so the orchestration treats agent liveness as a first-class systems problem rather than an exception path.

- **Heartbeats are file mtimes.** A chain agent appends to its retrieval ledger every two to three tool calls. This serves two purposes at once: it offloads context, and it produces an observable file-system heartbeat. Three minutes without a file change is treated as death.
- **Timeouts are tiered and enforced by killing.** Per-node caps are 40, 20 and 15 minutes for phases A, B and C; a phase round caps at 90 minutes and a 30-topic batch targets two hours. On timeout the node is killed — not diagnosed, not retried within the round.
- **Idempotence is by artifact existence.** A topic whose record exists is done and is never touched again, regardless of any status file; status metadata is explicitly distrusted in favour of the file system.

- **Timed-out work is not failed work.** A killed topic is *not* written to the failure ledger. It produced no record, so the next scan returns it to the pending set naturally. Only topics that failed validation after repair are recorded as failures — the ledger means “needs a human”, not “did not finish”.
- **Resumption is per-step, not per-topic.** Every phase begins with a breakpoint inventory that skips completed steps, so a killed topic resumes rather than restarting.

The batch loop follows from these: the pending set is computed as $\text{taxonomy} \cap \neg \text{disk} \cap \neg \text{failure-ledger}$, 30 topics are advanced through phase A, then B, then C, and the loop repeats without waiting for stragglers.

5.10 Allocating model capability

Because the phases differ in what they demand, model capability is allocated rather than applied uniformly:

Role	Dominant demand	Allocation
Phase B (merge)	cross-document same/different judgment	strongest available
Phase A (scan, gating)	domain breadth; the candidate list caps everything downstream	strong
Chain closed-loop agent	search planning, evidence discrimination <i>and</i> schema compliance	strong; low temperature
Red-team review	critical checking; one-shot	inexpensive relative to the re-work it prevents
Phase C (finalization)	mechanical checking; scripts carry most of the load	mid-tier

Two different kinds of model use. It is worth separating them, since they are often conflated. *Compilation* is performed by tool-using agents in an agent harness — they search the web, fetch pages, run verification scripts, read and write files, and dispatch sub-agents; the roles above refer to these. *Runtime and auxiliary services* — query intent decomposition, gap detection, chain assembly and duplicate adjudication — are ordinary API calls to a separate, cheaper chat model behind a single client abstraction, with temperature and model selectable per call site (low temperature and a retention-biased instruction for duplicate adjudication, for instance). The compiler needs agency; the runtime services need throughput. The specification records one explicit warning here: the chain agent must *not* be downgraded on the reasoning that asset verification is routine, because the same session goes on to emit the schema-conformant record. Closing the loop removed the option of using a cheap model for the mechanical segment — a real and acknowledged cost of the design.

5.11 The Retrieval Ledger

Within the chain agent, evidence gathering is where factual grounding is enforced, and it is the part of the system we consider most consequential. Segment A is where the system’s factual grounding is enforced, and it is the part we consider most consequential. Every chain research document must terminate in a mandatory **retrieval log**. The specification instructs floors of at least 10 web searches and 8 page fetches for the paradigm-research segment and 6 and 8 for the asset-research segment; the mechanical validator checks that the ledger section is present and well-formed, but does not itself compare the recorded counts against those floors, so compliance is instructed rather than enforced. In a random sample of 400 topic directories, documents carrying a parsable ledger record a median of 10 searches and 6 fetches, and 46% meet both instructed floors. Enforcing the counts in the validator is a small change we have not yet made. The log is not a summary count. It has four parts:

1. a **per-query table** recording every search issued, its hit count, the key systems/papers/terms it surfaced, and *which section of the document the result was used in* — so an unused query is visibly unused;
2. a **fetch and script table** recording every page fetched and every verification script invocation, with the extracted evidence quoted;
3. a **negative-results table** listing candidates that were found and *deliberately excluded*, with the exclusion rationale and the log line the candidate came from;

4. a **registry-lookup record** showing which existing capabilities the research reused rather than re-mining.

Repository claims are not taken on the model’s word: a verification script queries each candidate GitHub repository and returns its real star count, license and README text, which is quoted into the log. A paper-fetch script does the same for literature. A chain document that cannot ground a claim is required to mark it explicitly as unverified rather than assert it.

Table 2 reproduces an excerpt of a real ledger. Its most unusual feature is that it records *failure*: queries that returned nothing useful, and candidates that were rejected. A generated document that only ever reports successful retrieval is indistinguishable from a fabricated one; a ledger with honest negative entries is not.

Table 2: Excerpt from the retrieval ledger of a real chain research document (`evaluation_metric_definition`, chain 01). Translated from the stored Chinese. Note the recorded failures — the mechanism’s credibility rests on them.

#	Query / action	Used in	Outcome
<i>Search log</i>			
1	RLBench success_criteria task.py programmatic success determination PyRep	§1, §8	5 hits; weak sources
2	RLBench _success_criteria TaskStatus method stepjam github	—	5 hits; <i>no direct hit</i>
3	MetaWorld success flag termination condition manipulation benchmark	—	5 hits; <i>no direct hit</i>
4	LIBERO benchmark BDDL goal condition success MuJoCo	§1, §8	BDDL format confirmed
<i>Fetch / script log</i>			
1	check_github.py stepjam/RLBench	§1, §10, §11	1802 stars, License: Other; README quoted verbatim
3	check_github.py Farama-Foundation/Metaworld	§1, §10, §11	1859 stars, MIT; Gymnasium API confirmed
4	registry_lookup.py "RLBench success_criteria"	§5–§7	23 hits / 8 vector; 10 existing capabilities reused
<i>Negative results (found, then excluded)</i>			
—	ALFWorld task-level success determination	—	text-embodied, not simulation manipulation; belongs to sibling topic
—	Real-robot multimodal success detection	—	no ground truth on real hardware; out of chain scope

This regime is being rolled out across the corpus rather than applied retroactively. In a random sample of 220 topic directories (706 chain documents), 43.6% carry a full retrieval ledger, and those that do average 9.1 recorded searches and 6.1 recorded fetches in the paradigm-research segment alone; 225 of the 706 sampled chain documents (31.9%) carry a negative-results table. Documents compiled under the earlier regime remain in the artifact and are scheduled for re-compilation.

The intermediate evidence documents are retained as first-class artifacts, not discarded scaffolding: the compiler currently holds 5,510 chain research documents across 1,698 topic directories, an average of 3.24 chains per compiled topic. This means every capability in the graph is traceable back to the document that produced it, and that document to the queries that grounded it.

5.12 The structural validator

A task chain is a directed graph: steps carry **next** pointers, and branch/merge steps are permitted. LLM output is fluent but routinely produces graphs that are subtly illegal — unreachable steps, cycles, references to capability identifiers that do not exist. The validator checks this with three classical graph algorithms:

1. **In-degree analysis** to locate entry nodes. A chain with no zero-in-degree node is almost always cyclic.
2. **DFS/BFS reachability** from the entry set. Any unvisited node is orphaned and reported.
3. **Tri-colour marking** (white/grey/black) for strict cycle detection: encountering a grey node during traversal is a back edge, hence a cycle.

Beyond graph structure, it enforces roughly twenty additional rules: bidirectional consistency of capability parent/child declarations (if p lists c as a child, c must list p as parent), identifier format, asset reference integrity, and a platform-asset blacklist that forbids attaching generic platforms (ROS, PyTorch) directly to a specific algorithmic capability — a rule that exists because without it the implementation graph degenerates into everything pointing at ROS. Rules are two-tier: `ERROR` aborts the compilation unit with a non-zero exit; `WARN` is recorded but does not block.

5.13 Registry admission and global deduplication

Because compilation is per-topic and independent, the same capability (“inverse kinematics solving”) will be independently rediscovered by many topics. Registry admission is the linker: it decides whether an entity arriving from a compiled topic record is new, is an existing entity, or is a conflict. As described in §5.8, this runs asynchronously in the watchdog; the compiling agents only ever run it in check mode.

Hard conflicts. If an identifier is already occupied by an entity of a different type or a different upstream repository, the entire write is rejected. There is no partial admission.

Suspect detection — primary path. The current primary path is embedding-based. Each candidate and each incumbent are encoded into two views: *identity* (name and aliases) and *concept* (description). With s_{id} and s_{con} the respective cosine similarities, the combined score is

$$s = 0.5 s_{\text{id}} + 0.5 s_{\text{con}},$$

and a pair is flagged as a duplicate suspect when

$$s \geq 0.86 \quad \text{or} \quad \max(s_{\text{id}}, s_{\text{con}}) \geq 0.95.$$

Both thresholds were set empirically against a snapshot of roughly 9,099 capability vectors, in which the 95th percentile of *legitimate* nearest-neighbour similarity was ≈ 0.863 : the threshold sits at the background noise ceiling rather than being chosen by intuition.

We are deliberately careful about what this threshold is for. It is a **recall-oriented filter** whose job is to ensure that genuine duplicates reach adjudication, and it is tuned accordingly — it is expected to admit a substantial number of non-duplicates. We make no claim here that the similarity score is a good *decision* rule; the design assumes it is not, which is why adjudication is a separate, deliberately conservative stage rather than an automatic merge. Quantifying the gate’s precision and the marginal contribution of the adjudication stage is evaluation we have not yet run (§11).

Suspect detection — fallback path. When the GPU vector service is unavailable, the system degrades to a two-gate lexical path: description similarity ≥ 0.30 admits a pair to the candidate pool, but pairs in the grey band $[0.30, 0.35)$ additionally require token-set Jaccard ≥ 0.34 on names. This second gate was added after an audit found that a single 0.30 threshold misclassified approximately half of the grey band as duplicates.

Adjudication. Flagged pairs enter a ledger and are resolved by a two-stage chain. A non-LLM triage pass clears only *obvious non-duplicates* and never auto-merges; it is deliberately biased toward retention. Remaining pairs go to an LLM adjudicator in small batches (≤ 5 pairs, $T = 0.1$) with an explicit instruction to answer “different” under uncertainty, because a false merge is far more expensive than a false retention — it destroys information that must then be re-derived. Confirmed merges are executed by a merge routine that rewrites all references, unions aliases, re-renders, re-validates and re-synchronizes, with a hard guard against merging a parent-child pair.

An earlier design deferred adjudication to a background process; the backlog grew to 2,527 open suspects and was never processed. The pipeline now settles its own suspects at the end of each compilation unit. The ledger today holds 1,206 pairs, of which 1,116 are resolved, 39 merged, and 43 open. Because every entry records the similarity score, the metric that produced it, the adjudicating agent and the written rationale for the decision, the ledger is in effect a labelled dataset for duplicate detection in this domain, and we intend to release it.

Identity. Capability identifiers were originally derived from names (`CAP_<name>`), which meant that renaming an entity changed its identity and invalidated every reference to it. Identifiers are now opaque surrogate keys (`CAP_<8-12 hex>`) and names are a mutable display field, with renaming implemented as a single atomic operation covering reference rewriting, re-rendering, re-validation and registry resynchronization. This is the same argument as separating a database primary key from a business key, and the same argument as symbol tables in a compiler: identity must not be spelled.

5.14 Design rationale: what we rejected and why

Several of the choices above have obvious-looking alternatives. We record the reasoning, including where the reason is a constraint rather than a principle.

Why blocking gates rather than a quality score. An obvious alternative is to admit everything and attach a confidence score, letting consumers filter. We rejected this because the artifact’s value depends on invariants a consumer cannot cheaply re-check: that identifiers resolve, that chains are acyclic, that an entity appears once. A soft score gives a downstream planner permission to assume none of them. Blocking gates make the invariants true by construction; the price is that a single unfixable defect discards a whole compilation unit, which we accept because recompiling one topic is cheap and repairing a corrupted global registry is not.

Why registry admission is asynchronous rather than part of the transaction. The compiler analogy would put linking at the end of each translation unit. We do not, because 30 topics compile concurrently and a synchronous registry write would serialize them on a global lock — and because a registry-side failure would then be able to stall compilation entirely. Moving admission into a separate watchdog decouples the two, at the cost of the registry lagging the topic records and of the “everything admitted is valid” invariant being enforced by the watchdog rather than by the producing agent (§5.8).

Why the compilation unit is a topic, not a document or a chain. A per-document unit would make provenance simpler but produces no boundaries: two documents about neighbouring subjects generate overlapping capabilities with nothing to arbitrate between them. A per-chain unit is too small to see its own siblings. The topic is the smallest unit at which the taxonomy can supply *negative* scope — “these adjacent topics exist, do not absorb them” — which is what keeps the graph from collapsing into overlapping near-duplicates. This is the same reason compilers scope at the translation unit rather than the statement.

Why an LLM adjudicates duplicates rather than a threshold. Purely threshold-based merging is fast and wrong in an asymmetric way: a false merge destroys a distinction that then has to be re-derived, whereas a false retention leaves two entries that a later pass can still merge. Because the costs are asymmetric, the decision procedure is deliberately biased toward retention, and the threshold is used only to decide *what to look at* (§5.13). The residual cost of this policy is visible in our reuse statistics — some of the capabilities we count as distinct are probably not.

Why surrogate keys. Deriving identifiers from names couples identity to presentation: renaming becomes a graph-wide migration. This is the primary-key/business-key separation from database design and the symbol-table argument from compilers, and we adopted it only after the name-derived scheme had already made one rename expensive. The qualification we would add, having been bitten by it, is in §5.6: surrogate keys are correct for storage and reference and actively harmful as the input to any stage that must make a semantic judgment, so the point at which names become keys is itself a design decision.

Why an exact tensor index rather than FAISS or Milvus. This one is a constraint, not a principle: neither offers a supported GPU build on the team’s Windows deployment target. Having been forced into a hand-built exact index, we found the trade favourable at our scale — exact recall rather than approximate, and $O(1)$ insert and delete via swap-remove, which matters because the artifact is recompiled continuously rather than rebuilt in epochs. We would expect to revisit this well before the index outgrows a single device.

Why rank fusion rather than score fusion. Lexical and dense scores are not on comparable scales, and normalizing them introduces a tuning parameter that has to be re-tuned whenever either route changes. Reciprocal rank fusion [6] avoids the question. Two different K values are used because the coarse and fine stages have pools that differ by an order of magnitude and want different rank discrimination.

Why per-facet embeddings rather than one vector per entity. A single vector forces the name, the concept, the hierarchy position and the implementation into one average, where the strongest signal dominates and the others are diluted — which is precisely wrong for a corpus where users alternate between “what is MoveIt” and “how do I do X”. Splitting into facets costs index size (roughly $6\times$ for capabilities) and buys intent-conditioned weighting. The decisive argument was that three of the six facets are free: they are synthesized from graph structure, so the compiled artifact pays for its own retrieval improvements (§6.3).

6 Graph Representation

ENTROPY BOX does not store a single homogeneous graph. It stores several typed graphs over a shared entity space, because the questions they answer are different and conflating them destroys precision.

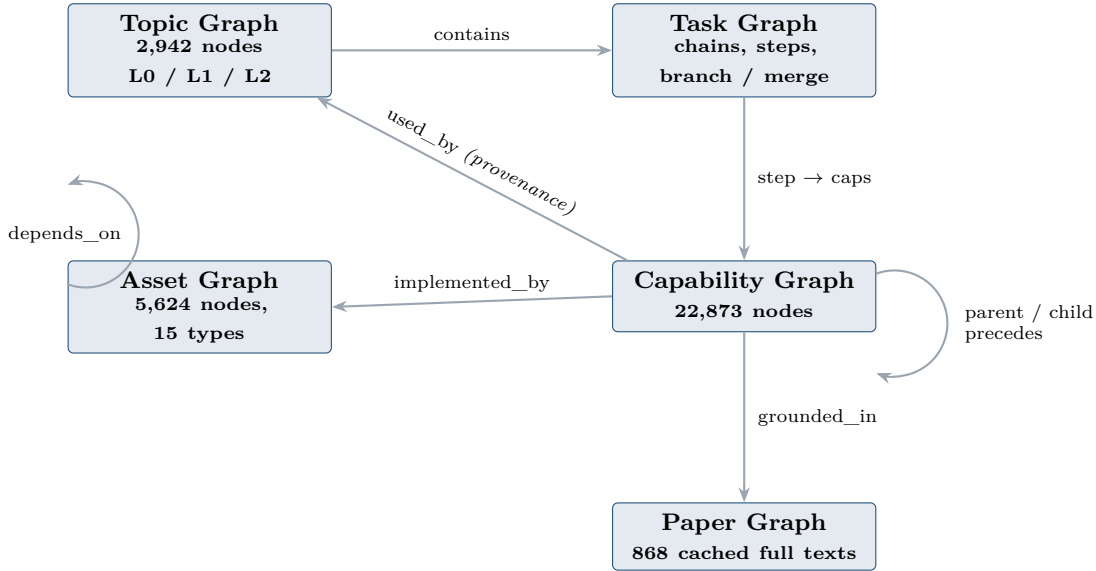


Figure 5: The multi-graph schema. Separating capability hierarchy, capability ordering and capability implementation into distinct edge families is what makes graph expansion useful at retrieval time: an expansion can follow exactly the relation the query intent implies.

6.1 Entities

Capability. A capability is an *ability*, not a tool: a named, typed unit of engineering competence. Each record carries an opaque identifier, an English Title-Case canonical name, a mandatory Chinese name, an alias list, and a description written to a fixed template covering method, object, prerequisites, inputs, outputs, constraints, performance envelope, applicability, and explicitly the *boundary against sibling capabilities*. That last requirement is not cosmetic: sibling-boundary text is what makes deduplication tractable, because near-duplicate capabilities differ precisely in their boundaries.

Capabilities are levelled: 43 at L1 (paradigm families), 5,576 at L2 (engineering paradigms) and 17,213 at L3 (concrete techniques); 41 records predate the levelling scheme and are unlevelled.

Asset. An asset is a *thing you can actually obtain*: a repository, framework, package, pretrained model, dataset, simulator, benchmark, robot platform, or paper. One GitHub repository is exactly one asset; the canonical name uses upstream spelling. Table 3 gives the distribution.

Table 3: Asset type distribution ($n = 5,624$).

Type	n	Type	n	Type	n
tool	1,633	dataset	567	benchmark	96
framework	1,251	ros_package	198	library	7
model	753	paper	166	software	5
package	687	robot	125	code	4
		simulator	114	hardware / other	6

Task chain. A task chain is an ordered, possibly branching graph of steps that accomplishes something. Each step declares the capabilities it requires. Branch steps carry an explicit **condition**; merge steps are flagged. Chains are the bridge between “what exists” and “what to do”.

6.2 Edge families

Edge family	Semantics	Count
parent/child	capability specialization	24,529
precedes/preceded_by	ordering within chains	43,973
cap→asset	implemented by	31,803
cap→topic (used_by)	provenance / reuse	36,713
context records	co-step and successor context	104,325

17,729 capabilities have a parent and 5,622 have children, giving a genuinely deep hierarchy rather than a flat tag cloud. 22,501 capabilities participate in at least one ordering edge.

6.3 Semantic views

A single vector per entity is a poor representation, because the name, the concept, the functional signature, the position in the hierarchy and the implementation are different kinds of information that dilute each other when averaged. ENTROPY BOX instead decomposes each entity into independently embedded **semantic views**: six for capabilities (*identity, concept, function, relation, context, implementation*), five for assets, three for topics.

Crucially, three of the six capability views — relation, context and implementation — are synthesized *directly from the graph* at zero LLM cost: the relation view is the entity’s hierarchy neighbourhood, the context view its dependency neighbourhood, the implementation view its asset set. The function view is extracted from the description by pattern matching the structured “input / dependency / output / constraint” fields that the description template mandates. This is a concrete payoff of compiling to structure: the graph *is* additional retrieval signal, obtainable for free.

The live index contains 174,189 view vectors (2560-d, fp16, Qwen3-Embedding-4B): 137,240 for capabilities, 28,120 for assets, 8,829 for topics.

7 Runtime

The runtime is a consumer of the compiled artifact. Its job is not to understand robotics; it is to find the right region of the graph and assemble it.

7.1 Retrieval

Figure 6 shows the retrieval stack. Its ordering constraint is that **LLM intent decomposition happens before all recall**, so that both the lexical and dense routes retrieve against the decomposed sub-queries rather than the raw string.

Intent-conditioned view weighting. The query classifier assigns one of six intents and selects a corresponding weight profile over the six capability views (Table 4). Precedence is Relation > Context > Implementation > Function > Identity > Concept; entity-name queries are detected either by matching the alias/term index or by short-ASCII-name shape.

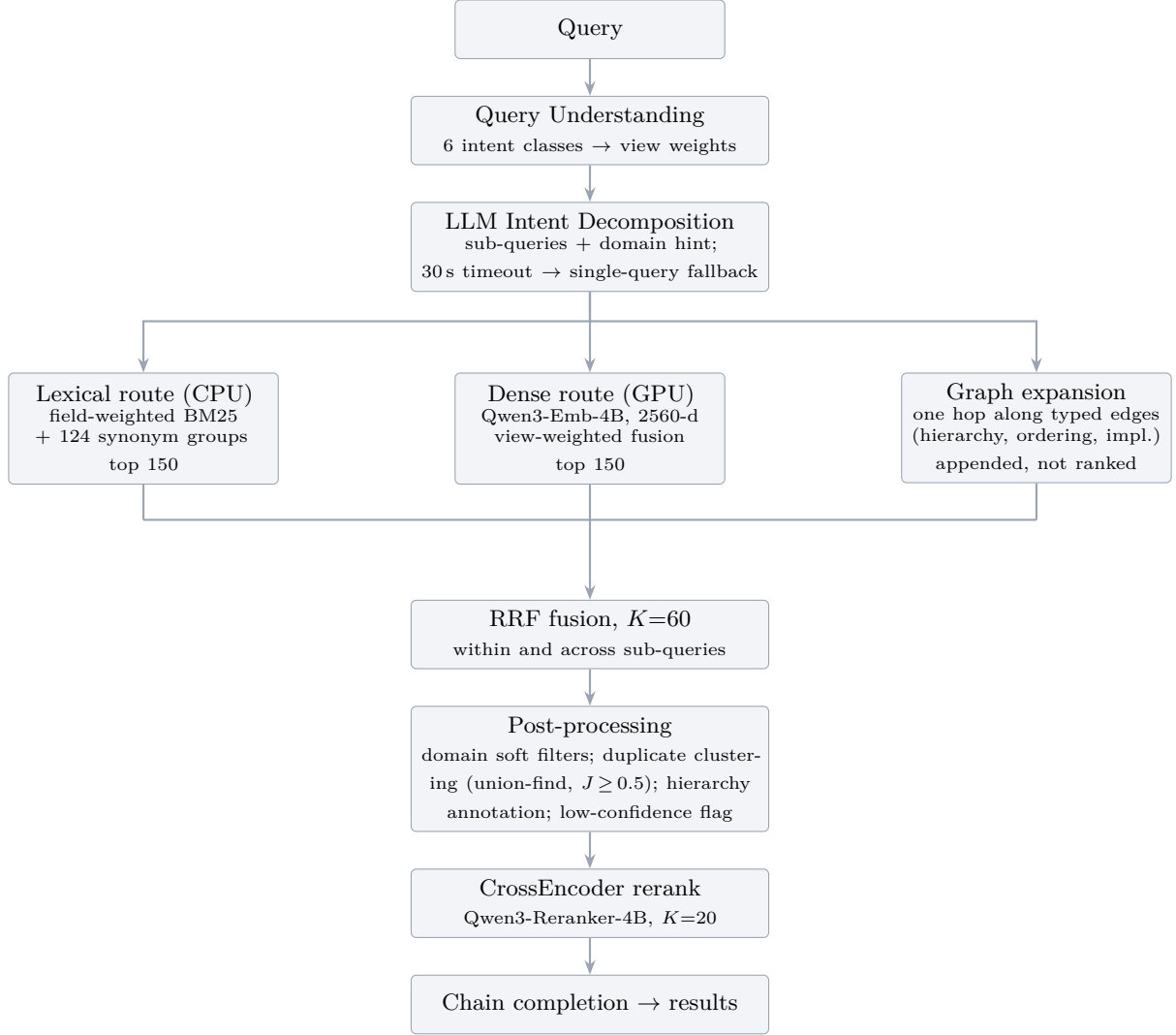


Figure 6: Three-layer hybrid retrieval. Each signal is scored exactly once; fusion happens only at the RRF layer, and the final ordering is determined solely by the cross-encoder.

Lexical route. Per-field BM25 ($k_1=1.5$, $b=0.75$) with independent indices so that document-frequency statistics of one field do not contaminate another. Name carries weight 5.0; exact hits add +2.0; contiguous phrase hits add $1.5 \times$ field weight; exact repository matches add 4.0; character-trigram matching provides spelling tolerance. A 124-group robotics synonym dictionary (e.g. IK $\leftrightarrow$ Inverse Kinematics) is applied symmetrically to queries and documents with a token weight of 1.5, which is the main lever for short Chinese queries where lexical overlap with English canonical names is zero.

Dense route. Qwen3-Embedding-4B, 2560 native dimensions, fp16 on GPU, asymmetric retrieval convention (instruction prefix on the query side only). The index is a hand-built exact PyTorch index: all vectors live in one L2-normalized GPU tensor, so cosine similarity is a single matrix multiply. This was a deliberate trade — neither FAISS-GPU nor Milvus-GPU is available on the team’s Windows deployment target — but it also buys exact 100% recall (not approximate) and $O(1)$ insert/delete via swap-remove, which matters because the artifact is continuously recompiled.

Fusion. Lexical and dense scores are not commensurable, so fusion is by reciprocal rank [6] rather than weighted sum, with two parameter regimes: $K=60$ at the coarse stage (large pool, ~ 150 per route) and $K=20$ at the fine stage (small pool), where sharper rank discrimination is wanted. Ties are broken deterministically by dense score, then lexical score, then identifier, so results are reproducible.

Graph expansion. For capability hits, a one-hop expansion follows hierarchy, ordering and implementation edges. Expanded results are appended after the ranked results and do not consume top- k slots — they

Table 4: Intent-conditioned semantic-view weight profiles (capability scope, normalized).

Intent	identity	concept	function	relation	context	implementation
Identity	0.55	0.15	0.05	0.05	0.05	0.15
Function	0.10	0.15	0.50	0.05	0.10	0.10
Relation	0.10	0.10	0.05	0.55	0.15	0.05
Context	0.05	0.10	0.10	0.15	0.55	0.05
Implementation	0.15	0.10	0.10	0.05	0.05	0.55

are structural context, not competing answers.

7.2 Assembly and gap detection

The runtime’s distinguishing endpoint is not search but *assembly*. Given a goal, the system retrieves a candidate pool across topics and asks an LLM to compose one or more executable chains from it, allowing branch and merge. Every capability identifier the model emits is validated against the pool: an identifier that does not exist is first fuzzy-matched, and if it still cannot be resolved it is converted into an explicit **gap proposal** rather than silently dropped.

This is the property that a RAG system structurally cannot provide. The answer to “how do I build this” is not a paragraph; it is a chain in which every node is labelled either *owned* (a real capability with real assets) or *gap* (a required capability nothing in the artifact implements). Gaps are then fed back as compilation targets, closing the loop in Figure 3.

8 Case Study: Compiling a_star_path_planning

We trace one real compilation unit end to end. The topic `a_star_path_planning` sits at L2 under Navigation. It is a useful case precisely because it looks trivial — “ A^* is a textbook algorithm” — and is not.

8.1 Input: evidence

The evidence planner produced a seed query matrix of ten seeds, among them:

```
A* path planning robot navigation occupancy grid
Hybrid A* Reeds-Shepp car-like vehicle path planning
Jump Point Search JPS pathfinding grid Harabor
Theta* any-angle path planning Daniel Nash line-of-sight
```

together with a synonym expansion table and an explicit boundary note excluding the sibling topic `cost_map_planning`. The resulting chain document enumerates the paradigm landscape, a representative-systems table with repository URLs and per-chain attribution, and a key-paper list.

8.2 Output: four non-equivalent chains

The compiler did *not* produce “one A^* capability”. It produced four chains distinguished by state-space representation and expansion rule:

Chain	State space	Expansion rule	Anchor
Grid A^*	2-D occupancy cell	4/8-connected	Hart et al. 1968 [14]
Hybrid A^*	continuous SE(2)	kinematic primitives, Reeds–Shepp	Dolgov et al. 2008 [9]
JPS	2-D occupancy cell	look-ahead + jumping rules	Harabor & Grastien 2011 [13]
Theta*	2-D occupancy cell	line-of-sight parent rewiring	Daniel et al. 2010 [7]

This distinction is exactly the thing a retrieval system does not give you and an engineer needs first. A car-like robot with non-holonomic constraints cannot use grid A^* at all; a query that returns “ A^* is a heuristic graph search with $f(n) = g(n) + h(n)$ ” is not merely incomplete, it is operationally misleading.

8.3 Output: capabilities

Compilation admitted 20 capabilities for this topic: 4 at L2, one per paradigm, and 16 at L3. Table 5 lists them with the assets the implementation graph attaches to each. Note that the four L2 capabilities are siblings, not synonyms — each carries the boundary text that distinguishes it from the other three, which is what allowed them to survive deduplication as distinct entities.

Table 5: Compiled capabilities for the `a_star_path_planning` topic and the assets the implementation graph attaches to each, as stored in the live artifact. Each capability is a distinct registry entity; the surrogate keys are omitted here for readability.

Level	Capability	Implementing assets
<i>Chain 1 — Grid A* (discrete occupancy grid, 4/8-connected)</i>		
L2	Grid Based A* Path Planning	nav2_smac_planner, nav2_theta_star_planner, Hart–Nilsson–Raphael 1968
L3	Discrete Grid A* Search	PythonRobotics, nav2_smac_planner, Hart–Nilsson–Raphael 1968
L3	Admissible Heuristic Selection for Grid A*	PythonRobotics, nav2_smac_planner
L3	Open/Closed Set Management on Grid	PythonRobotics, nav2_smac_planner
L3	Obstacle Map Inflation and Collision Check	MoveIt 2, Nav2, nav2_costmap_2d, nav2_smac_planner, PythonRobotics, SBPL
L3	Path Reconstruction via Parent-Index Backtracking	PythonRobotics, nav2_smac_planner
<i>Chain 2 — Hybrid A* (continuous SE(2), kinematic primitives)</i>		
L2	Continuous-State Hybrid A* Path Planning	Nav2, navfn, move_base, Dolgov et al. 2008
L3	Hybrid A* SE(2) State Node Expansion	PythonRobotics, nav2_smac_planner, kfreund/hybrid-a-star, Dolgov et al. 2008
L3	Reeds–Shepp Curve Shortcut Expansion	PythonRobotics, nav2_smac_planner, kfreund/hybrid-a-star
L3	Motion Primitive Steer Set Discretization	PythonRobotics, nav2_smac_planner, kfreund/hybrid-a-star
L3	Vehicle Footprint Continuous Collision Check	PyBullet, Box2D, Jolt Physics, PythonRobotics, nav2_smac_planner
L3	Hybrid A* 2D Reverse Heuristic	nav2_smac_planner
<i>Chain 3 — Jump Point Search (symmetry pruning on the same grid)</i>		
L2	Jump Point Search Grid A* Optimization	Harabor–Grastien 2011
L3	JPS Look-ahead Rule Pruning	hjaandroid/JumpPointSearch, qiao/PathFinding.js
L3	JPS Jumping Rule Recursive Expansion	hjaandroid/JumpPointSearch, qiao/PathFinding.js, Harabor–Grastien 2011
L3	Forced Neighbor Detection	hjaandroid/JumpPointSearch, qiao/PathFinding.js
<i>Chain 4 — Theta* (any-angle, line-of-sight rewiring)</i>		
L2	Any-Angle Theta* Path Planning	nav2_theta_star_planner, PythonRobotics, Daniel et al. 2010
L3	Theta* Line-of-Sight Bresenham Check	nav2_theta_star_planner, PythonRobotics, Daniel et al. 2010
L3	Theta* Parent Rewiring Path Compression	nav2_theta_star_planner, PythonRobotics, Daniel et al. 2010

8.4 What the compiled record actually contains

The stored description for *Grid Based A* Path Planning* is not a summary. Rendered from its template it declares: the procedure (obstacle inflation → open/closed set initialization → heuristic evaluation → 4/8-connected main loop → parent-index backtracking); the optimality condition ($f(n) = g(n) + h(n)$ is optimal when h is admissible); the applicability split (4-connected for Manhattan motion, 8-connected for omnidirectional); prerequisites (discrete grid map, start and goal); inputs (obstacle point set, endpoints, robot radius); outputs (grid waypoint sequence); constraints (admissible heuristic, single-source single-sink);

a performance envelope (millisecond scale on a 100×100 grid); and canonical implementations.

This is the difference the compiler makes. Each field is separately embedded as a semantic view, separately retrievable, and separately consumable by a downstream planner.

8.5 Reuse

Obstacle Map Inflation and Collision Check is a good illustration of the linker at work. It was minted once during this compilation and is now referenced by eight distinct L2 topics:

```
a_star_path_planning    adaptive_coverage_path
chance_constrained_navigation    configuration_space_planning
safety_margin_planning    topological_graph_representation
uncertainty_aware_planning    underwater_obstacle_avoidance
```

It links to six assets spanning navigation, manipulation and search-based planning stacks. A per-topic document generator would have written eight unrelated paragraphs; the compiler wrote one node with eight inbound provenance edges. Across the whole artifact, 6,533 capabilities (28.6%) are used by more than one topic, over 36,713 capability–topic usages.

9 Measurements and Evaluation Infrastructure

Table 6 reports the state of the live compiled artifact. We emphasize that these are inventory and structural statistics, not a benchmark result; §11 discusses what we do not yet measure.

Table 6: Compiled artifact statistics. Entity counts are a live registry snapshot (2026-07-29); edge counts are from the derived graph indices (2026-07-27) restricted to entities present in the live registry, so every edge endpoint resolves.

Category	Quantity	Value
Taxonomy	L0 domains	15
	L1 sub-domains	416
	L2 compilation units	2,511
Progress	L2 topics with a compiled record on disk	1,740 (69.3%)
	L2 topics represented in the graph indices	1,605 (63.9%)
	L2 topics not yet compiled	771
	Topic directories with evidence documents	1,698
	Chain evidence documents	5,510 (3.24 / topic)
Entities	Capabilities	22,873
	L1 / L2 / L3	43 / 5,576 / 17,213
	Assets	5,624 (15 types)
	Cached arXiv full texts	868
Edges	Hierarchy (parent→child)	24,529
	Ordering (precedes)	43,973
	Implementation (cap→asset)	31,803
	Provenance (cap→topic)	36,713
	Context records	104,325
Structure	Capabilities with a parent	17,729 (77.5%)
	Capabilities with children	5,622 (24.6%)
	Capabilities reused across topics	6,533 (28.6%)
Grounding	Capabilities with ≥ 1 asset	15,689 (68.6%)
	by level: L1 / L2 / L3	32.6% / 47.7% / 75.5%
	Mean assets per grounded capability	2.03
Index	Semantic view vectors	174,189
	capabilities / assets / topics	137,240 / 28,120 / 8,829
	Encoder	Qwen3-Embedding-4B, 2560-d, fp16
	Synonym groups	124
Quality control	Suspect pairs: resolved / merged / open	1,116 / 39 / 43
	Peak unadjudicated backlog (historical)	2,527

Coverage is a progress reading, not a ceiling. The compiler is running — the topic-record count rose by one during the measurement run that produced this section. At the time of measurement 1,740 of 2,511 L2 topics (69.3%) had a compiled record on disk, leaving 771 uncompiled.

Two coverage figures appear in Table 6 because they measure different things, and we report both rather than picking the flattering one: 1,740 records exist on disk, while the derived graph indices attribute capabilities to 1,605 distinct topics. The gap is index lag — records are written continuously and the indices are rebuilt periodically — and it is the same lag that explains the 11% discrepancy between the two independent amortization estimates in §9.2. We note additionally that the pipeline’s own run ledger reported a lower figure (1,081) than a direct disk scan; the ledger is a cache and the file system is authoritative, which is what the orchestration specification itself asserts.

Reading the numbers. Three of the remaining figures carry most of the signal.

First, **grounding rate rises with specificity**: 75.5% of L3 capabilities have at least one concrete implementing asset, versus 47.7% at L2 and 32.6% at L1. This is the expected and desirable shape — an abstract paradigm is not supposed to have a repository, a concrete technique is — and it is a sanity check on level assignment.

Second, **reuse is the metric that distinguishes a compiler from a document generator**. An earlier internal audit (2026-07) found that roughly 96% of capabilities were referenced by exactly one topic, i.e. the system was mostly producing per-topic documents that happened to share a schema. Under the current registry-candidate-injection and deduplication regime that figure has moved to 71.6% single-topic / 28.6% multi-topic. Reuse is the direct measure of whether the symbol table is doing its job, and it is the metric we optimize.

Third, **the suspect ledger is a control-system reading, not a defect count**. A backlog of 2,527 was not evidence of a dirty knowledge base; it was evidence of an unbounded queue with no consumer. Settling suspects inside each compilation transaction bounded the queue: 43 open pairs against 22,873 capabilities.

9.1 Evaluating the deduplication gate

The adjudicated duplicate ledger is a labelled dataset: every entry records the similarity score that flagged the pair, the metric that produced it, and the adjudicated outcome. Treating **merged** as “true duplicate” and **resolved** as “not a duplicate” gives 1,155 labelled pairs, which lets us ask a question the system’s design implicitly answers but never tested: once a pair has been flagged, *does the similarity score carry information about whether it is actually a duplicate?*

Scope, stated first. These pairs are exactly those that already passed the gate, so the measurement is conditional on being flagged. It does not estimate the gate’s overall discriminative power over all pairs, and we do not report it as such. It answers the narrower and operationally decisive question: should the score influence the decision *after* flagging?

Result. It should not. Table 7 reports precision at the operating threshold and the area under the ROC curve using similarity as the duplicate score, with bootstrap confidence intervals over 2,000 resamples.

Table 7: Deduplication gate evaluated on 1,155 adjudicated pairs. AUC uses the similarity score as a duplicate detector *within the flagged set*; 0.5 is chance.

Path	pairs	true dup.	precision	AUC	95% CI
Embedding (primary)	508	35	0.069	0.629	[0.531, 0.722]
Lexical (fallback)	647	4	0.006	0.454	[0.271, 0.764]

The embedding path carries weak but statistically detectable signal — its confidence interval excludes chance — and is nonetheless far too weak to serve as a decision rule: 93% of what it flags is not a duplicate. The lexical fallback path has a point estimate *below* chance, but with only four positive examples its interval spans almost the entire range; we report it for completeness and draw no conclusion from it beyond the obvious one, that a fallback path exercised this rarely cannot be validated from production data.

Tuning does not fix this. The natural response is to raise the threshold. Figure 7 shows what that buys. Moving the operating point from 0.86 to 0.94 raises precision from 0.069 to 0.126 — still one true duplicate in eight — while discarding half the true duplicates. Beyond 0.95 the curve becomes noise-dominated on single-digit counts. There is no operating point at which the score alone is a usable decision procedure.

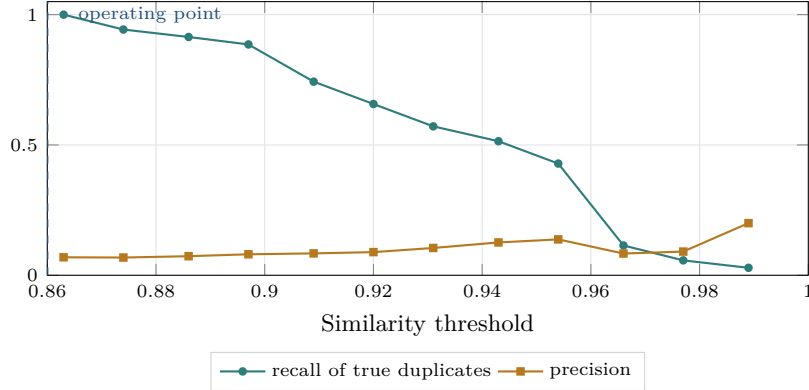


Figure 7: Threshold sweep for the embedding deduplication path within the flagged set. Raising the threshold trades away true duplicates far faster than it buys precision; past 0.95 the counts fall to single digits and the estimates become unstable. The design consequence is that the gate cannot be made into a decision rule by tuning.

What this justifies. The system’s two-stage design — a recall-oriented threshold followed by a conservative adjudication stage that is instructed to answer “different” under uncertainty — was originally adopted on the argument that false merges are irreversible. This measurement converts that argument from a design preference into a requirement. The adjudication stage is not a safety margin on top of a working classifier; it is doing essentially all of the discrimination. Any system that flags near-duplicate entities by embedding similarity and then *merges automatically above a threshold* would, at our operating point, be wrong more than nine times in ten.

We regard this as the most transferable empirical result in the paper, and note that it is only measurable because adjudication decisions were recorded with their scores and rationales rather than applied and forgotten.

9.2 Compilation output: structure, provenance and refusal

Three further measurements over the full set of 1,740 compiled topic records (the compiler was running during measurement; counts drift upward by a few records per hour).

Chain structure. The 1,740 records contain 5,756 task chains (3.31 per topic) with 11.07 steps per chain on average (median 10, p90 16). Of these, 22.8% contain a branch, 22.2% contain an explicit merge node, and **74.1% are purely linear**. An earlier internal audit had estimated 69%; the full-corpus figure is slightly worse. Since the schema supports branching and the validator does not require it, this is an extraction-incentive problem: the compiler produces the simplest structure that passes, and nothing rewards capturing genuine route divergence. It is the clearest quality deficit we can currently measure.

Provenance integrity. Every topic record cites the chain research documents it was assembled from. Because those documents are files, the citation can be checked mechanically. Of 5,545 chain-document references, **98.29% resolve to a file that exists**; 95 are broken, and 58 records (3.3%) carry no chain-document reference at all. Of the topic-level research document references, 5 are broken.

We propose this as a metric for LLM-constructed knowledge graphs generally. Structural validity is routinely checked; *provenance resolvability* — the fraction of an artifact’s own evidence citations that actually resolve — is not, and it is cheap to compute, hard to game, and directly diagnostic of whether a pipeline’s evidence layer is real or decorative. A system that cannot report it has no evidence layer to speak of.

Refusal to fabricate. The specification requires that an unverifiable field be marked unverified rather than filled in, and treats inventing a citation as a more serious failure than leaving a gap. That instruction

is followed: 1,105 unverified markers survive in the compiled artifact, distributed over 16.2% of topic records (mean 0.64 per record). We report this deliberately as a *positive* indicator. The alternative — zero unverified markers — would not indicate a better artifact; given what is actually verifiable about fast-moving open-source projects, it would indicate a pipeline that fills gaps rather than declaring them.

Deduplication, seen from the records. The 1,740 records contain 40,834 capability entries and 18,110 asset entries, which the registry collapses to 22,873 and 5,624 distinct entities respectively — a $1.79\times$ collapse for capabilities and $3.22\times$ for assets. The capability figure is an independent estimate of the amortization factor of §2.3, arrived at by counting record entries rather than provenance edges, and it agrees with the edge-derived estimate (1.61) to within 11%. The residual gap is provenance-edge lag: the graph indices are rebuilt periodically while records are written continuously.

9.3 Retrieval evaluation: the harness, and the state of its results

Retrieval quality cannot be inferred from inventory statistics, so we built a domain-specific evaluation suite. We describe its design here as a methodological contribution, and then state plainly that we do not currently have a valid result from it.

The golden set. 126 queries across eight intent classes (Table 8), written in the mixed Chinese/English register that actual users of a robotics knowledge base type. Three properties distinguish it from a naive relevance set.

First, **assertions come in three strengths**, because not every query has a single correct entity. **expect** names exact identifiers; **expect_any** accepts any member of a set, which is the honest form for a query like “detect the cup on the table and generate a grasp pose” that legitimately spans two entities; and **expect_match** is a structural predicate — for example, *a result anchored to topic `a_star_path_planning` whose text contains “heuristic”* — which is the only workable form for mechanism questions whose answer lives in a chain document or a step card rather than in an entity description.

Second, it carries **negative assertions**. **reject** and **reject_match** declare what must *not* be retrieved, and the false-recall rate is reported alongside recall. The most instructive of these takes a literal web-search query string lifted from a chain document’s retrieval ledger — text that looks exactly like a technical question — and asserts that it must not retrieve any retrieval-log section. That is a test that the evidence index does not leak its own provenance metadata back as content, and it is the kind of failure that only a system with a provenance layer can have.

Third, the set is **regenerated rather than maintained**. Identifiers are surrogate keys that change under registry rebuilds, so the checked-in golden file is a build product: the maintained source names entities by name, and a build step resolves names to current identifiers. This is the same names-versus-keys discipline as §5.6, applied to test data. Several entries carry annotations recording the audit that motivated them — for instance a query whose Chinese phrasing was found to be misclassified into the wrong intent class — so the suite accumulates regressions rather than only covering the happy path.

Table 8: Golden set composition ($n = 126$). Assertion counts overlap with intent classes; six queries are diagnostic only and carry no positive assertion.

Intent class	n	Assertion form	n
identity	34	expect (exact ids)	34
concept	25	expect_any (any-of set)	49
function	15	expect_match (predicate)	37
selection	15	reject (must not retrieve)	8
evidence	13	reject_match	5
io	10		
multihop	7		
negative	7		

The harness. The evaluation driver reports nDCG@10, recall@20, MRR, false-recall rate against the negative assertions, and latency at the 50th, 95th and 99th percentiles — broken down by intent class, which matters because the retrieval stack conditions its view weighting on exactly that classification (§7) and a stack that is strong on identity queries and weak on mechanism queries would otherwise average out to something unremarkable.

What we do not have. We have no valid current run. The most recent execution recorded a complete failure — all 126 queries failed with zero latency, the signature of the vector service being unavailable rather than of poor retrieval — and the last successful run predates the surrogate-key migration, covers only six queries, and asserts against identifiers that no longer exist. We therefore report no retrieval numbers in this paper. The apparatus is built and the gap is an execution gap rather than a research one; we will publish results, and the suite itself, separately.

Table 9: Retrieval evaluation: metrics the harness reports, awaiting a valid run. We show the empty table deliberately, to be unambiguous about what is and is not claimed in this paper.

Intent class	n	nDCG@10	recall@20	MRR	false-recall	p95 latency
identity	34	—	—	—	—	—
concept	25	—	—	—	—	—
function	15	—	—	—	—	—
selection	15	—	—	—	—	—
evidence	13	—	—	—	—	—
io / multihop / negative	24	—	—	—	—	—
overall	126	—	—	—	—	—

10 Discussion: What Entropy Box Is Not

It is easy to place a system like this in the wrong category, so we state the distinctions directly.

Not a search engine. A search engine returns documents ranked by relevance to a string. ENTROPY BOX returns typed entities with declared inputs, outputs, prerequisites and implementations, plus the edges between them. The output is intended for a program, not a reader.

Not a RAG system. RAG derives structure at query time and discards it. ENTROPY BOX derives structure at compile time and keeps it. A RAG system asked the same question a thousand times performs the analysis a thousand times and can give a thousand mutually inconsistent answers; a compiler performs it once and serves the same artifact. Concretely: our compilation of `a_star_path_planning` into four non-equivalent paradigms is a durable fact in the graph, not a lucky generation.

Not a vector database. A vector database is a component of our runtime, not the system. The interesting content of ENTROPY BOX is the typed edge families — hierarchy, ordering, implementation — which are what let graph expansion, chain completion and gap detection work. Vectors alone give you neighbours; they do not give you *precedes*.

Not an execution ontology. ENTROPY BOX does not model action semantics, object affordances or robot state in the sense of KnowRob [26] or IEEE 1872 [15]. It models engineering decisions. We view these as complementary layers and consider the interface between them — compiled capability $\rightarrow$ executable action description — an important direction, and one that RCDS (§10.1) is intended to make mechanical rather than manual.

Not classical knowledge compilation. As stated in §1, we do not claim the tractability guarantees of the OBDD/d-DNNF line of work [8]. Our compilation target is a typed property graph, and our “tractability” claim is empirical and much weaker: queries that would require multi-step LLM derivation over raw text become graph traversals plus retrieval.

10.1 Beyond the graph: a description standard and a trust ladder

A compiled capability graph answers “what exists and what implements it”. It does not yet answer “can I trust this, and will it compose with what I already have”. Two mechanisms extend the artifact toward those questions.

RCDS. The Robot Capability Description Schema is a proposed specification for describing a robot capability in composable terms: declared inputs and outputs, service-level expectations, certification status, and the interface fields required for composition. The purpose is mechanical: with a shared schema, a planner can automatically check whether two capabilities are data-flow compatible before proposing that they be chained, instead of leaving that check to a human reading two READMEs. The schema is what turns a graph of descriptions into a graph of *composable* units. We regard standardization here as the durable contribution — a shared description format outlives any particular graph built with it.

A three-level evidence ladder. Capability claims are not equally trustworthy, and collapsing them onto one axis is how knowledge bases lose credibility. We distinguish three levels: **L1**, benchmark performance on a standard dataset, which reflects theoretical capability; **L2**, results in a simulation environment such as Isaac or MuJoCo; and **L3**, validated deployment on real hardware in a production setting, including sustained operating data. The point is not that L3 supersedes L1 — for many purposes a dataset number is exactly what you want — but that a consumer of the graph must be able to see *which kind* of evidence backs a claim. This ladder is orthogonal to the compilation gates: those govern structural admission, the ladder governs epistemic weight.

System layering. The compiler is one layer of three. Layer 1 is the compiler described in this paper. Layer 2 is the engine: retrieval, reasoning, task decomposition and planning over the artifact (§7). Layer 3 is the application surface — question answering, agent and MCP access, and capability composition. This paper is about Layer 1, because Layer 1 is what the other two are impossible without.

The positioning claim. We contend that source-level knowledge compilation is a missing *infrastructure layer* for robot foundation models, sitting between the open literature and the model in the same way that a package registry sits between source code and an application. Models will keep improving. The knowledge they need to be grounded in will keep being published as prose. Something has to compile it.

11 Limitations and Future Work

We list what does not work, because a system paper that reports only successes is not useful to anyone.

Provenance coverage is partial. The retrieval-ledger regime described in §5.11 — enforced search/fetch floors, per-query accounting, negative-results tables, script-verified repositories — is the current standard, but roughly 44% of chain documents in our sample carry a full ledger. The remainder were compiled under an earlier regime that did not enforce it. Those records are structurally valid and were not fabricated, but they are not *auditable* to the same standard, and we do not claim they are. Re-compiling them under the current regime is queued work, not a research problem.

Structural validity is guaranteed; factual accuracy is not. The structural and registry gates guarantee that the graph is well-formed, that identifiers resolve, that there are no cycles and that entities are globally unique. They do not guarantee that a capability’s stated performance envelope is correct. The evidence ledger makes factual claims *traceable* — which is a precondition for verification, not a substitute for it. We do not yet run systematic post-hoc factual audit against primary sources, and expert spot-checking at 22,873 capabilities is not tractable manually. Automated cross-verification (does the cited repository actually expose the claimed module? does the cited paper actually state the claimed bound?) is the most important open item.

Grounding coverage is incomplete. 31.4% of capabilities have no attached asset and carry only an inline implementation hint. Those hints are not structured, not comparable, and not reusable. The gap is concentrated at higher abstraction levels, which is partly correct by design and partly a real hole.

Reuse is still low. 28.6% multi-topic reuse is a large improvement over the 4% of the earlier audit but is still far from where a well-linked graph should be. Some of the residual is genuine (many L3 techniques really are topic-specific); some is unresolved near-duplication that our deliberately conservative adjudication policy declines to merge.

Chains are under-branched. Measured over the full corpus, 74.1% of task chains are purely linear (§9.2), even though the schema supports branching, merging and sub-chain references. The validator does not require branching and the extractor does not volunteer it, so the compiler emits the simplest structure that passes. This is a prompt-and-incentive problem rather than a schema problem, and it is the quality deficit we would attack first.

The central comparison has not been run. We have argued for compilation and quantified its amortization (§2), but we have not demonstrated that a compiled artifact produces better answers than a strong query-time baseline. The experiment we owe the reader is a head-to-head against naive RAG and GraphRAG over the same source corpus, measuring answer correctness, hallucination rate, coverage, end-to-end latency and total cost of ownership including build cost — the last of which is the honest way to account for a system that front-loads work. We do not report it because it does not exist yet, and reporting a comparison we have not run would be worse than reporting none.

What stands between us and that experiment is smaller than it was. The domain-specific golden set exists (§9.3); what is missing is a valid run of it, and the baselines to run beside it. The remaining hard requirement is instrumentation: the pipeline records no per-topic compilation time or token cost, so build-side economics — the exact quantity that decides whether front-loading is worth it — cannot currently be reported at all. That is a logging change, not a research question, and it is the single cheapest thing we could do to strengthen the argument of this paper.

The gate evaluation is small and conditional. §9.1 rests on 35 positive examples for the embedding path and four for the lexical fallback, and is conditioned on pairs that were already flagged. The embedding result is stable enough to act on — the confidence interval excludes chance and the threshold sweep is monotone over the range that matters — but the lexical result is not a finding, and neither speaks to the gate’s recall: we cannot measure the duplicates the gate never flagged, because nothing labelled them. A proper recall estimate requires adjudicating a random sample of *unflagged* pairs, which we have not done.

Encoder is not domain-adapted. Qwen3-Embedding-4B is a general model. Robotics terminology (dense acronyms, repository names, mathematical notation) is plausibly not well separated in its embedding space, which bounds both retrieval and deduplication precision. Domain-adaptive fine-tuning on the compiled artifact itself is a natural next step — and one the artifact uniquely enables, since it provides typed positive and hard-negative pairs at scale.

Description templating introduces noise. Enforcing a description template improves parseability but induces formulaic phrasing, which adds lexical and semantic noise to retrieval. Balancing template compliance against descriptive distinctiveness is unresolved.

Synonym expansion is exact-group only. The 124 synonym groups match true synonyms. Broad-concept expansion (SLAM $\rightarrow$ loop closure, pose graph, scan matching) is not yet implemented; the graph contains exactly this information and should supply it.

Multilingual scope. Entities carry both English and Chinese canonical names by schema, so the graph is bilingual by construction and further languages require only additional name fields, not schema or relation changes. Full bilingual coverage of long-form descriptions is not yet complete.

12 Conclusion

The constraint on embodied AI is increasingly the availability of *structured* engineering knowledge rather than model capacity. Retrieval-augmented generation addresses the symptom — getting relevant text in front of a model — while leaving the underlying problem untouched: the structure is re-derived on every query, in prose, without identity or persistence.

ENTROPY BOX takes the other approach. It compiles heterogeneous robotics sources into a persistent typed artifact through a gated pipeline, treats the LLM as an untrusted parser subject to admission control, and exposes the artifact to a runtime that can assemble executable capability chains and name what is missing. The artifact as of this snapshot — 22,873 capabilities, 5,624 assets, over 100,000 typed edges and 174,189 semantic-view vectors, drawn from 1,605 topics, with 1,430 more queued and the compiler still running — is large enough to indicate that the approach scales. It is also, deliberately, a snapshot

of something in motion: the claim we are making is about the pipeline, not about the size of yesterday’s output.

Compile Once, Execute Everywhere.

Availability

The paper, the public indices of the compiled artifact, the retrieval golden set, the adjudicated deduplication ledger and the measurement script are published at <https://chenli-yy.github.io/entropy-box-public/>, together with interactive views of the topic graph and the capability dependency network and one topic reproduced in full. All data is released under CC BY 4.0 and the code under MIT.

A read-only retrieval API over the evidence corpus is open and requires no registration or client software:

```
POST <api-base>/api/evidence/search
{ "query": "...", "top_k": 10, "mode": "hybrid", "rerank": true }
```

The current API base URL is listed on the project page above. The same endpoint is packaged as an Agent Skill declaration and as an MCP stdio server, so that external agents can query the artifact directly. The open endpoint currently exposes concept-level evidence retrieval; complex task retrieval, chain assembly and node-level graph queries are implemented but not yet exposed as an API, and are reachable through the web interface.

We intend to release the taxonomy, the RCDS specification, an English snapshot of the capability and asset registries, and a retrieval benchmark.

Acknowledgements

The compilation pipeline, the artifact and this paper are the work of a very small team. We thank the maintainers of the open-source robotics projects that constitute most of the asset graph — in particular Nav2, MoveIt, PythonRobotics and the wider ROS ecosystem — whose documentation quality is what made automated compilation feasible at all.

References

- [1] M. Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv:2204.01691*, 2022.
- [2] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives. DBpedia: A Nucleus for a Web of Open Data. In *ISWC/ASWC*, 2007.
- [3] M. Beetz, L. Mösenlechner, and M. Tenorth. CRAM — A Cognitive Robot Abstract Machine for Everyday Manipulation in Human Environments. In *IEEE/RSJ IROS*, 2010.
- [4] A. Brohan et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv:2307.15818*, 2023.
- [5] D. Coleman, I. Şucan, S. Chitta, and N. Correll. Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study. *Journal of Software Engineering for Robotics*, 5(1), 2014.
- [6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *ACM SIGIR*, 2009.
- [7] K. Daniel, A. Nash, S. Koenig, and A. Felner. Theta*: Any-Angle Path Planning on Grids. *Journal of Artificial Intelligence Research*, 39:533–579, 2010.
- [8] A. Darwiche and P. Marquis. A Knowledge Compilation Map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002.
- [9] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel. Practical Search Techniques in Path Planning for Autonomous Driving. In *AAAI Workshop on Search Techniques in Robotics*, 2008. (Extended: *Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments*, IJRR 29(5), 2010.)
- [10] D. Driess et al. PaLM-E: An Embodied Multimodal Language Model. In *ICML*, 2023.

- [11] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv:2404.16130*, 2024.
- [12] Y. Gao et al. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997*, 2023.
- [13] D. Harabor and A. Grastien. Online Graph Pruning for Pathfinding on Grid Maps. In *AAAI*, 2011.
- [14] P. E. Hart, N. J. Nilsson, and B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [15] IEEE Standards Association. IEEE 1872-2015: Standard Ontologies for Robotics and Automation. IEEE, 2015.
- [16] M. Y. Jaradeh et al. Open Research Knowledge Graph: Next Generation Infrastructure for Semantic Scholarly Knowledge. In *K-CAP*, 2019.
- [17] V. Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP*, 2020.
- [18] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- [19] S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The Marathon 2: A Navigation System. In *IEEE/RSJ IROS*, 2020.
- [20] R. Nogueira and K. Cho. Passage Re-ranking with BERT. *arXiv:1901.04085*, 2019.
- [21] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv:2310.08864*, 2023.
- [22] M. Quigley et al. ROS: An Open-Source Robot Operating System. In *ICRA Workshop on Open Source Software*, 2009.
- [23] Qwen Team. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv:2506.05176*, 2025.
- [24] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.
- [25] A. Saxena et al. RoboBrain: Large-Scale Knowledge Engine for Robots. *arXiv:1412.0691*, 2014.
- [26] M. Tenorth and M. Beetz. KnowRob: A Knowledge Processing Infrastructure for Cognition-Enabled Robots. *International Journal of Robotics Research*, 32(5):566–590, 2013.
- [27] D. Vrandečić and M. Krötzsch. Wikidata: A Free Collaborative Knowledge Base. *Communications of the ACM*, 57(10):78–85, 2014.
- [28] M. Waibel et al. RoboEarth. *IEEE Robotics & Automation Magazine*, 18(2):69–82, 2011.